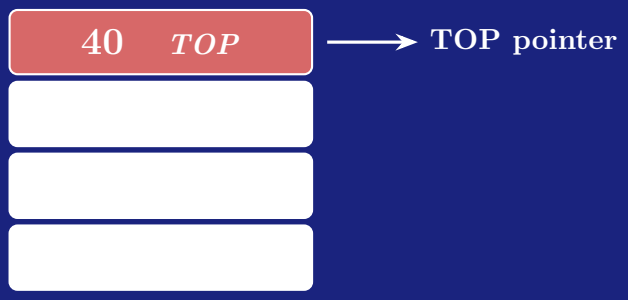
STACK

Data Structure — Complete Reference

Basic to Advanced · Array · Linked List · Vector · Problems



LIFO Principle · Push · Pop · Peek · Applications

Contents

1 Introduction to Stack **3**

1.1 Definition and LIFO Principle . . . 3

1.2 LIFO Visualisation — Corrected Step-by-Step . . . 3

1.3 Core Operations . . . 4

2 Implementation 1 — Stack Using `std::vector` **5**

2.1 Overview . . . 5

3 Implementation 2 — Stack Using a Fixed Array **7**

3.1 Internal Design . . . 7

3.2 Memory Layout — Used vs. Unused Slots . . . 7

3.3 Complete Annotated Implementation . . . 7

3.3.1 Operation Trace Table . . . 10

4 Implementation 3 — Stack Using a Linked List **11**

4.1 Why Dynamic Allocation? . . . 11

4.2 Node Structure . . . 11

4.2.1 Linked Chain Visualisation . . . 11

4.3 Complete Annotated Implementation . . . 11

4.4 Array vs. Linked-List Stack . . . 14

5 Two Stacks in One Array **15**

5.1 Motivation and Design . . . 15

5.2 Diagram — Accurate Index Positions . . . 15

5.3 Complete Corrected Implementation . . . 15

6 Assignment Problems **19**

6.1 Problem 1 — Reverse a Stack . . . 19

6.1.1 Problem Statement 19
6.1.2 Algorithm: Three-Stack Pour Method 19
6.2 Problem 2 — Remove Adjacent Duplicate Characters 21
6.2.1 Problem Statement 21
6.2.2 Trace for “`aabba`” 21
6.3 Problem 3 — Balanced Parentheses 22
6.3.1 Problem Statement 22
6.3.2 Algorithm 22
6.3.3 Trace for `"([])"` 22
6.4 Problem 4 — Decimal to Binary Conversion 24
6.4.1 Algorithm 24
6.5 Problem 5 — Postfix Expression Evaluation 25
6.5.1 Notation Overview 25
6.5.2 Algorithm 25

7 Complexity Summary **28**

8 Quick Reference **29**
8.1 Unified Method Names Across All Implementations 29
8.2 Implementation Comparison 29
8.3 Interview Pattern Reference 31
8.4 Key Formulae 31

9 Summary of All Implementations and Problems **33**

1. Introduction to Stack

1.1 Definition and LIFO Principle

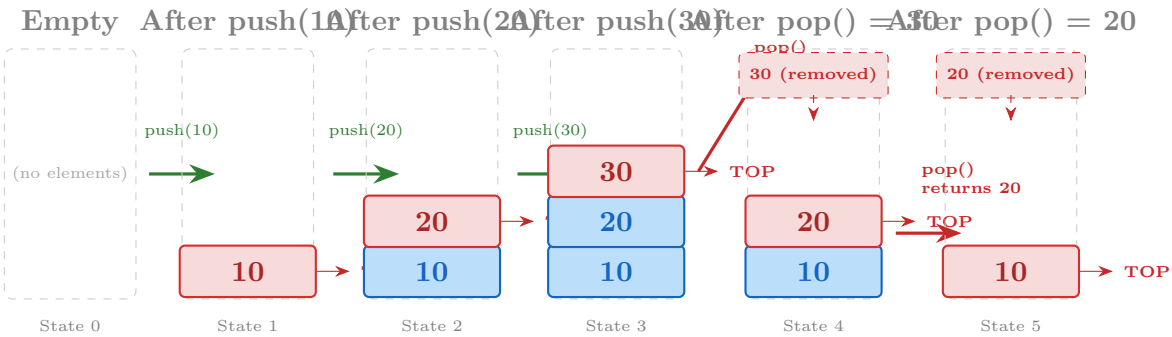
A **Stack** is a linear data structure that follows the **LIFO** (Last In, First Out) ordering principle. The element inserted most recently is always the first to be removed.

Formal Definition

A stack is an ordered collection of elements in which insertions and deletions are permitted only at one end, called the **top**. Insertions are termed **push** and deletions are termed **pop**. At any point the element at the top is accessible via **peek**.

1.2 LIFO Visualisation — Corrected Step-by-Step

The diagram below shows three push operations followed by two pop operations. Each frame is a *complete, consistent snapshot* of the stack at that instant. Removed elements are shown *above* the stack only in the frame where they leave; they do *not* appear inside the stack in later frames.



Reading the Diagram

- Removed elements appear *above* the stack boundary (dashed box) only in the frame where they are popped. They are *absent* from all subsequent frames.
- The **TOP** arrow always points to the current topmost live element.
- Push order: $10 \to 20 \to 30$. Pop order: $30 \to 20 \to 10$ (LIFO confirmed).

1.3 Core Operations

Operation	Signature	Time	Description
PUSH	`push(val)`	O(1)	Insert `val` at the top
POP	`pop()`	O(1)	Remove and return top element
PEEK	`peek()`	O(1)	Return top element without removal
isEmpty	`isEmpty()`	O(1)	True when no elements are present
isFull	`isFull()`	O(1)	True when capacity is exhausted (array only)

2. Implementation 1 — Stack Using `std::vector`

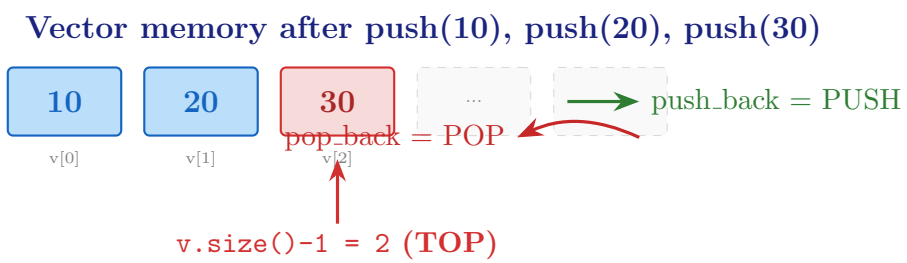
2.1 Overview

The `std::vector`-based stack is the simplest implementation. A vector is a dynamically resizing array, so no fixed capacity is needed. The *back* of the vector serves as the top of the stack.

```cpp
#include <iostream>
#include <vector>
using namespace std;

class Stack {
    vector<int> v;   // back of vector = top of stack

public:
    // PUSH: append to back -- O(1) amortised
    void push(int val) {
        v.push_back(val);
    }

    // POP: remove from back -- O(1)
    void pop() {
        if (isEmpty()) {
            cerr << "Stack underflow: pop() on empty stack." << endl;
            return;
        }
        v.pop_back();
    }

    // PEEK: read back element without removal -- O(1)
    int peek() const {
        if (isEmpty()) {
            cerr << "Stack underflow: peek() on empty stack." << endl;
            return -1;   // sentinel value
```

```
        }
        return v[v.size() - 1];
    }

    // isEmpty: true when vector holds no elements
    bool isEmpty() const {
        return v.empty();
    }

    int size() const { return static_cast<int>(v.size()); }
};

int main() {
    Stack s;
    s.push(10);    // v = [10]
    s.push(20);    // v = [10, 20]
    s.push(30);    // v = [10, 20, 30]

    while (!s.isEmpty()) {
        cout << s.peek() << " ";    // prints: 30 20 10
        s.pop();
    }
    cout << endl;
    return 0;
}
```

Listing 2.1: Stack using `std::vector` — annotated



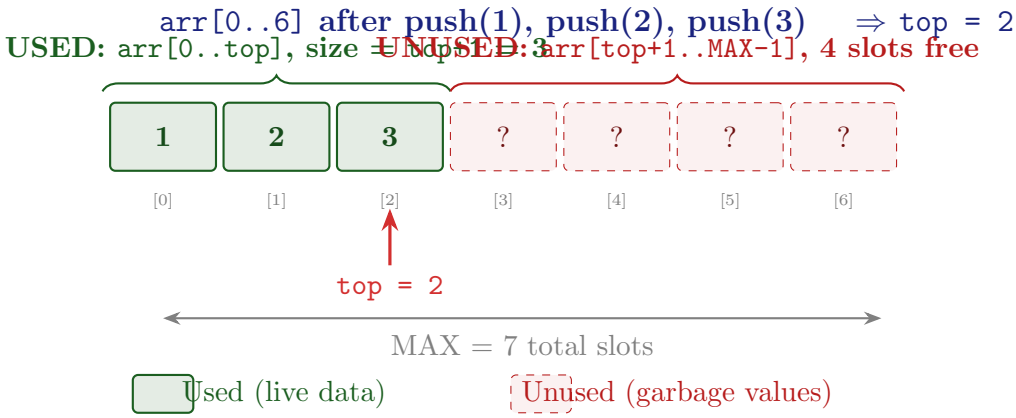
3. Implementation 2 — Stack Using a Fixed Array

3.1 Internal Design

An array-based stack allocates a fixed block of memory of size `MAX` and uses an integer `top` as the index of the current topmost element.

$$\textbf{Invariants:} \quad \texttt{top} = -1 \text{ (empty)} \quad 0 \leq \texttt{top} \leq \texttt{MAX} - 1 \text{ (elements present)}$$
$$\texttt{top} = \texttt{MAX} - 1 \text{ (full)}$$

3.2 Memory Layout — Used vs. Unused Slots



3.3 Complete Annotated Implementation

```
#include <iostream>
#include <stdexcept>
using namespace std;

class Stack {
    int* arr;     // dynamically allocated storage
```

```
    int  top;     // index of topmost element; -1 when empty
    int  cap;     // maximum number of elements

public:
    // Constructor
    explicit Stack(int capacity) : cap(capacity), top(-1) {
        arr = new int[cap];
    }

    // Destructor -- must free heap memory
    ~Stack() { delete[] arr; }

    //         isEmpty

    bool isEmpty() const { return top == -1; }

    //         isFull

    bool isFull()  const { return top == cap - 1; }

    //         size

    int size() const { return top + 1; }

    //         push

    // Check overflow BEFORE modifying top
    void push(int val) {
        if (isFull()) {
            throw overflow_error("Stack overflow: capacity exhausted.");
        }
        arr[++top] = val;   // pre-increment: top becomes the new index
    }

    //         pop

    // Check underflow BEFORE decrementing top (fix applied here)
    int pop() {
        if (isEmpty()) {           // guard FIRST -- never touch top if empty
            throw underflow_error("Stack underflow: stack is empty.");
        }
```

```
        return arr[top--];          // return value, then post-decrement top
    }

    //        peek


    int peek() const {
        if (isEmpty()) {
            throw underflow_error("Stack underflow: stack is empty.");
        }
        return arr[top];            // read only -- top is unchanged
    }
};

int main() {
    Stack s(10);

    s.push(1);    // arr=[1,...]   top=0
    s.push(2);    // arr=[1,2,..]  top=1
    s.push(3);    // arr=[1,2,3]   top=2

    cout << "peek = " << s.peek() << endl;  // 3, stack unchanged

    while (!s.isEmpty()) {
        cout << s.pop() << " ";  // 3 2 1
    }
    cout << endl;

    // Underflow guard demo
    try {
        s.pop();
    } catch (const underflow_error& e) {
        cout << "Caught: " << e.what() << endl;
    }
    return 0;
}
```

Listing 3.1: Array-based stack — complete implementation with underflow guard

3.3.1 Operation Trace Table

Step	`Call`	top before	`top after`	Effect
0	`(init)`	–	`-1`	arr is allocated; all slots garbage
1	`push(1)`	-1	`0`	arr[0]=1
2	`push(2)`	0	`1`	arr[1]=2
3	`push(3)`	1	`2`	arr[2]=3
4	`peek()`	2	`2`	returns arr[2]=3; top unchanged
5	`pop()`	2	`1`	returns 3; arr[2] now unused
6	`pop()`	1	`0`	returns 2; arr[1] now unused
7	`pop()`	0	`-1`	returns 1; stack empty
8	`pop()`	-1	`--`	throws `underflow_error`

4. Implementation 3 — Stack Using a Linked List

4.1 Why Dynamic Allocation?

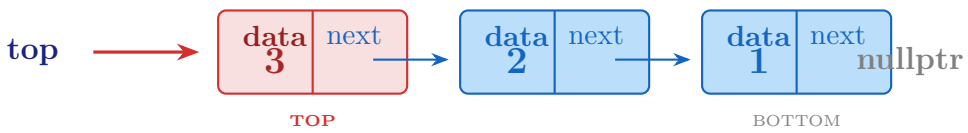
An array-based stack requires the capacity to be known at construction time. A linked-list stack is *fully dynamic*: each element is stored in a heap-allocated node, and the stack grows or shrinks by one node per operation.

4.2 Node Structure

```
struct Node {
    int   data;
    Node* next;   // points to the node BELOW this one (
        previously the top)

    explicit Node(int val) : data(val), next(nullptr) {}
};
```

Listing 4.1: Node definition

4.2.1 Linked Chain Visualisation



4.3 Complete Annotated Implementation

```
#include <iostream>
#include <stdexcept>
using namespace std;

```

```cpp
struct Node {
    int   data;
    Node* next;
    explicit Node(int val) : data(val), next(nullptr) {}
};

class Stack {
    Node* top;   // nullptr == empty stack
    int   sz;

public:
    Stack() : top(nullptr), sz(0) {}

    // Destructor -- pop every node to release all heap memory
    ~Stack() {
        while (!isEmpty()) pop();
    }

    bool isEmpty() const { return top == nullptr; }
    int  size()    const { return sz; }

    //        push

    void push(int val) {
        Node* n  = new Node(val);  // allocate new node
        n->next  = top;            // link new node to current top
        top      = n;              // advance top pointer
        ++sz;
    }

    //        pop

    // Returns value of removed node; uses delete to prevent memory leak
    int pop() {
        if (isEmpty())
            throw underflow_error("Stack underflow: stack is empty.");
        Node* tmp  = top;          // remember node to delete
        int   val  = tmp->data;    // save its value
        top        = top->next;    // move top to next node
        delete tmp;                // *** explicit delete: no memory leak ***
        --sz;
        return val;
    }

```

```
    //          peek


    // Safety guard added: check top != nullptr before dereferencing
    int peek() const {
        if (isEmpty())              // *** null-pointer guard ***
            throw underflow_error("Stack underflow: stack is empty.");
        return top->data;
    }
};

int main() {
    Stack s;
    s.push(1);    // [1]
    s.push(2);    // [2->1]
    s.push(3);    // [3->2->1]

    cout << "peek = " << s.peek() << endl;  // 3

    while (!s.isEmpty())
        cout << s.pop() << " ";    // 3 2 1
    cout << endl;
    return 0;
}
```

Listing 4.2: Linked-list stack with null-pointer guard and explicit delete

1. **Null-pointer guard in `peek()`:** The original code accessed `top->data` without checking whether `top` is `nullptr`. On an empty stack this causes a segmentation fault. The guard `if (isEmpty())` is inserted first.
2. **Explicit `delete` in `pop()`:** Every heap-allocated `Node` must be freed with `delete`. Omitting `delete` causes a memory leak: the operating system never reclaims that memory until the process exits.

4.4 Array vs. Linked-List Stack

Feature	Array	Linked List
Capacity	Fixed at construction	Unbounded (limited by RAM)
Overflow risk	Yes, when `top == cap-1`	No
Extra memory per element	None	One pointer (`next`)
Cache locality	Excellent (contiguous)	Poor (scattered heap nodes)
Capacity known upfront	Required	Not required
All operation times	O(1)	O(1)

5. Two Stacks in One Array

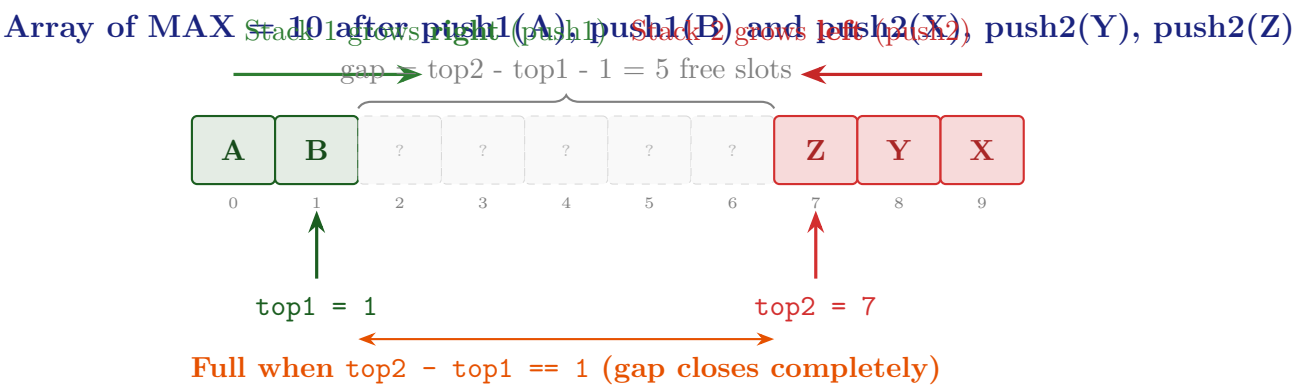
5.1 Motivation and Design

Naïvely splitting one array into two halves of size $\lfloor MAX/2 \rfloor$ wastes space when one stack is large and the other is small. A smarter approach lets Stack 1 grow from index 0 upward while Stack 2 grows from index $MAX-1$ downward. The two stacks share the unused middle.

Full condition: `top2 − top1 == 1` (stacks meet, no gap left)

Stack 1 empty: `top1 == -1` **Stack 2 empty:** `top2 == MAX`

5.2 Diagram — Accurate Index Positions



5.3 Complete Corrected Implementation

```
#include <iostream>
#include <stdexcept>
using namespace std;

class TwoStack {
    int* arr;
```

```
    int  cap;
    int  top1;   // Stack 1: grows left-to-right; init = -1
    int  top2;   // Stack 2: grows right-to-left; init = cap

public:
    explicit TwoStack(int capacity)
        : cap(capacity), top1(-1), top2(capacity)
    {
        arr = new int[cap];
    }

    ~TwoStack() { delete[] arr; }

    //        Stack-1 operations


    void push1(int val) {
        if (top2 - top1 == 1)
            throw overflow_error("Both stacks full.");
        arr[++top1] = val;
    }

    int pop1() {
        if (top1 < 0)
            throw underflow_error("Stack 1 is empty.");
        int x = arr[top1];    // save value first
        --top1;
        return x;             // *** FIX: return x, not top1
           ***
    }

    int peek1() const {
        if (top1 < 0)
            throw underflow_error("Stack 1 is empty.");
        return arr[top1];
    }

    bool empty1() const { return top1 < 0; }

    //        Stack-2 operations


    void push2(int val) {
        if (top2 - top1 == 1)
            throw overflow_error("Both stacks full.");
        arr[--top2] = val;
    }

    int pop2() {
        if (top2 >= cap)
            throw underflow_error("Stack 2 is empty.");
```

```
        int x = arr[top2];     // save value first
        ++top2;
        return x;              // *** verified: returns x, not index ***
    }

    int peek2() const {
        if (top2 >= cap)
            throw underflow_error("Stack 2 is empty.");
        return arr[top2];
    }

    bool empty2() const { return top2 >= cap; }
};

int main() {
    TwoStack ts(10);

    ts.push1(1); ts.push1(2); ts.push1(3);
    ts.push2(10); ts.push2(20); ts.push2(30);

    cout << "pop1: " << ts.pop1() << "\n";  // 3
    cout << "pop2: " << ts.pop2() << "\n";  // 30
    cout << "peek1: " << ts.peek1() << "\n"; // 2
    cout << "peek2: " << ts.peek2() << "\n"; // 20
    return 0;
}
```

Listing 5.1: TwoStack — with `pop1()` bug fixed and `pop2()` verified

Bug Fix: `pop1()` Returned the Wrong Value

Original (incorrect):

```
int pop1() {
    if (top1 >= 0) {
        int x = arr[top1];
        top1--;
        return top1;    // BUG: returns the decremented INDEX, not the data
    }
    return -1;
}
```

Corrected:

```
int pop1() {
    if (top1 < 0) throw underflow_error("Stack 1 is empty.");
    int x = arr[top1--];    // save value, then decrement
    return x;               // return the saved DATA value
}
```

The same pattern (`return x`) is applied to `pop2()` for consistency.

6. Assignment Problems

6.1 Problem 1 — Reverse a Stack

6.1.1 Problem Statement

Reverse the order of all elements in a stack using only stack operations and temporary stacks (no arrays, no direct index access).

Example: Stack top-to-bottom: $[2, 1]$. After reversal: $[1, 2]$ (1 is now at the top).

6.1.2 Algorithm: Three-Stack Pour Method

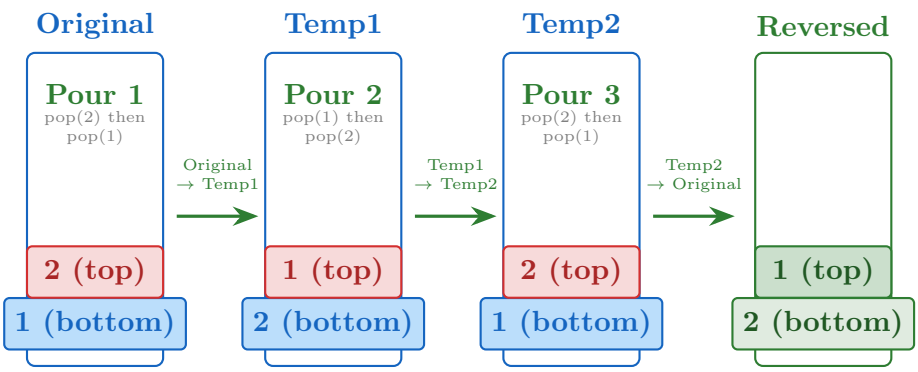
Why Three Pours?

Each complete "pour" from one stack into another *reverses* the order (LIFO means the first element out of the source becomes the last element in the destination).

Pour	Effect	Order in destination
Original → Temp1	Reversal #1	Reversed from original
Temp1 → Temp2	Reversal #2	Restored to original order
Temp2 → Original	Reversal #3	Reversed from original = **final goal**

After an *odd* number of reversals the sequence is reversed.

Three-Stack Reverse: input top-to-bottom = $[2, 1]$, goal = $[1, 2]$



```cpp
#include <iostream>
using namespace std;

class Stack {
    int* arr; int top; int cap;
public:
    explicit Stack(int c) : cap(c), top(-1) { arr = new int[cap]; }
    ~Stack() { delete[] arr; }
    bool isEmpty()  const { return top == -1; }
    bool isFull()   const { return top == cap - 1; }
    void push(int v){ if (!isFull())  arr[++top] = v; }
    int  pop()      { return isEmpty() ? -1 : arr[top--]; }
    int  peek()     const {
        if (isEmpty()) throw underflow_error("Empty.");
        return arr[top];
    }

    // Reverse in place using two temporary stacks
    void reverseStack() {
        Stack temp1(cap);
        Stack temp2(cap);

        // Pour 1: Original -> Temp1  (reversal #1)
        while (!isEmpty())       temp1.push(pop());

        // Pour 2: Temp1 -> Temp2    (reversal #2, restores original order)
        while (!temp1.isEmpty()) temp2.push(temp1.pop());

        // Pour 3: Temp2 -> Original (reversal #3, final reversed order)
        while (!temp2.isEmpty()) push(temp2.pop());
    }
};

int main() {
    Stack s(10);
    s.push(1);            // bottom
    s.push(2);            // top
    s.reverseStack();
    // Now: 1 is top, 2 is bottom
    while (!s.isEmpty())
        cout << s.pop() << " ";  // prints: 1 2
    cout << endl;
    return 0;
}
```

Listing 6.1: Reverse a stack using three pours

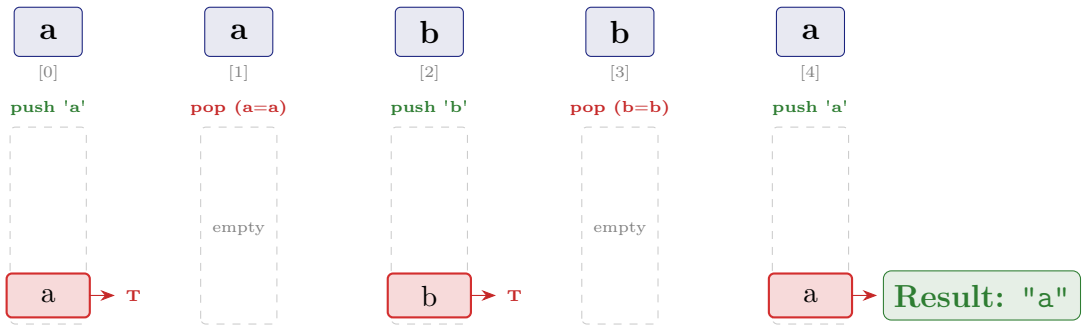
6.2 Problem 2 — Remove Adjacent Duplicate Characters

6.2.1 Problem Statement

Given a string, remove all adjacent duplicate characters until none remain.

Input	Output
aabbcc	(empty string)
abba	(empty string)
aabba	a
abcd	abcd

6.2.2 Trace for "aabba"




```cpp
#include <iostream>
#include <string>
using namespace std;

string removeAdjacentDuplicates(const string& word) {
    int n = word.length();
    // Use a char array as a manual stack (size = n worst case)
    char* stk = new char[n];
    int top = -1;

    for (int i = 0; i < n; i++) {
        // If current char equals top of stack: adjacent duplicate
        if (top >= 0 && stk[top] == word[i]) {
            --top;            // pop (cancel both the stored and current char)
        } else {
            stk[++top] = word[i];    // push
        }
    }

```

```
    // Build result from bottom (index 0) to top
    string result(stk, stk + top + 1);
    delete[] stk;
    return result;
}

int main() {
    string tests[] = {"aabbcc", "abba", "aabba", "abcd"};
    for (const string& t : tests)
        cout << '"' << t << "\" -> \""
             << removeAdjacentDuplicates(t) << '"' << endl;
    return 0;
}
```

Listing 6.2: Remove adjacent duplicates using a stack

6.3 Problem 3 — Balanced Parentheses

6.3.1 Problem Statement

Determine whether the brackets in an expression are correctly matched and nested.

Expression	Result
`([])`	Balanced
`([)]`	Unbalanced (mismatch)
`{[`	Unbalanced (unclosed openers)

6.3.2 Algorithm

1. For each character in the expression:
   - Opening bracket `(` `{` `[` → push it.
   - Closing bracket `)` `}` `]` → if the stack is empty, return **Unbalanced**; otherwise pop the top and verify the pair matches; if the pair does not match, return **Unbalanced**.
2. After scanning all characters: if the stack is empty → **Balanced**; otherwise → **Unbalanced**.

6.3.3 Trace for `"([])"`

```
#include <iostream>
#include <string>
```

```cpp
using namespace std;

bool isBalanced(const string& expr) {
    int n = expr.length();
    char* stk = new char[n];
    int top = -1;

    for (int i = 0; i < n; i++) {
        char c = expr[i];

        // Opening bracket: push onto stack
        if (c == '(' || c == '{' || c == '[') {
            stk[++top] = c;
        }
        // Closing bracket: match against top of stack
        else if (c == ')' || c == '}' || c == ']') {
            if (top < 0) {                // stack empty: no opener to match
                delete[] stk;
                return false;
            }
            char t = stk[top--];          // pop the top opener

            // Corrected condition syntax (no spaces inside quotes)
            if (c == ')' && t != '(') { delete[] stk; return false; }
            if (c == '}' && t != '{') { delete[] stk; return false; }
            if (c == ']' && t != '[') { delete[] stk; return false; }
        }
    }

    bool balanced = (top < 0);  // balanced iff no unmatched openers remain
    delete[] stk;
    return balanced;
}

int main() {
    string tests[] = {"([{}])", "([)]", "{[", "()", "([{})()"};
    for (const string& t : tests)
        cout << '"' << t << "\" -> "
             << (isBalanced(t) ? "Balanced" : "Unbalanced") << endl;
    return 0;
}
```

Listing 6.3: Balanced parentheses — corrected C++ syntax

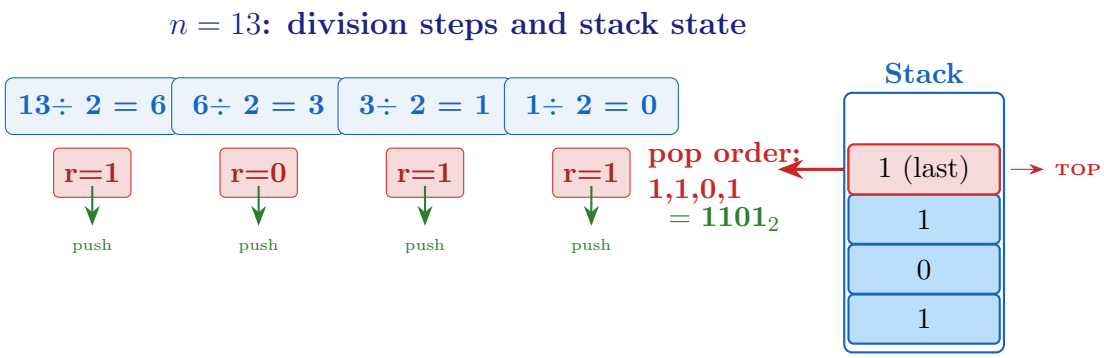
Syntax Fix: Bracket Condition Formatting

The original code contained whitespace inside character literals such as `expre[i]==') )` and `s.peek()==:('`, which are not valid C++. The corrected form uses single quotes with no extra spaces: `c == ')'` and `t == '('`.

6.4 Problem 4 — Decimal to Binary Conversion

6.4.1 Algorithm

Repeatedly divide by 2 and collect remainders. The binary representation is the remainders read in *reverse* order. A stack provides the reversal for free via LIFO.




```cpp
#include <iostream>
using namespace std;

void decimalToBinary(int n) {
    if (n == 0) { cout << "0\n"; return; }

    int stk[40];    // 40 bits is sufficient for a 32-bit integer
    int top = -1;

    while (n > 0) {
        stk[++top] = n % 2;    // push remainder (LSB first)
        n /= 2;
    }

    // Pop order = MSB first = correct binary representation
    while (top >= 0)
        cout << stk[top--];
    cout << endl;
}

int main() {
    decimalToBinary(13);    // 1101
    decimalToBinary(0);     // 0
    decimalToBinary(255);   // 11111111
```

```
    return 0;
}
```

Listing 6.4: Decimal to binary using a stack

6.5 Problem 5 — Postfix Expression Evaluation

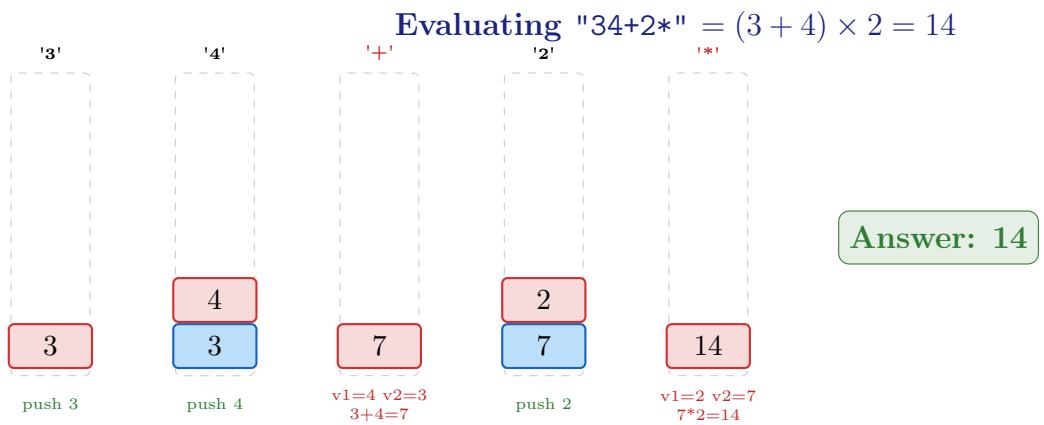
6.5.1 Notation Overview

Notation	Example	Notes
Infix	`3 + 4`	Operator between operands; needs parentheses for precedence
Postfix	`3 4 +`	Operator after operands; no parentheses needed

6.5.2 Algorithm

Scan left to right:

- Digit → push it.
- Operator → pop `val1` (right operand), pop `val2` (left operand), compute `val2 OP val1`, push result.




```
#include <iostream>
#include <string>
#include <cctype>    // isspace
using namespace std;

int evaluatePostfix(const string& expr) {
    int stk[50], top = -1;

    for (char c : expr) {
        if (c >= '0' && c <= '9') {
```

```
            stk[++top] = c - '0';        // digit: push numeric value
        }
        else if (c=='+' || c=='-' || c=='*' || c=='/') {
            if (top < 1)
                throw invalid_argument("Malformed postfix expression.");
            int val1 = stk[top--];       // right operand (popped first)
            int val2 = stk[top--];       // left  operand (popped second)
            int result = 0;
            switch (c) {
                case '+': result = val2 + val1; break;
                case '-': result = val2 - val1; break;  // val2 - val1 (order matters)
                case '*': result = val2 * val1; break;
                case '/': result = val2 / val1; break;
            }
            stk[++top] = result;
        }
        else if (isspace(static_cast<unsigned char>(c))) {
            continue;   // skip spaces -- allows "3 4 + 2 *" style input
        }
        // Any other character (e.g. letters) is silently ignored
    }

    if (top != 0)
        throw invalid_argument("Malformed postfix expression.");
    return stk[0];
}

int main() {
    cout << evaluatePostfix("34+2*")  << endl;  // 14  : (3+4)*2
    cout << evaluatePostfix("52-3*")  << endl;  // 9   : (5-2)*3
    cout << evaluatePostfix("23*4+")  << endl;  // 10  : (2*3)+4
    return 0;
}
```

Listing 6.5: Postfix evaluation — corrected operand order

Operand Order: Why `val2 OP val1`?

The top of the stack holds the *most recently pushed* value. For the expression `5 2 -` the stack before `-` contains $[5\ (\text{bottom}), 2\ (\text{top})]$. Popping gives `val1 = 2` then `val2 = 5`. The correct result is $5 - 2 = 3 =$ `val2` $-$ `val1`. Using `val1 - val2` would give -3, which is wrong.

7. Complexity Summary

Operation	Array	Linked List	Vector	Reason
push()	O(1)	O(1)	O(1)*	Only touch top pointer
pop()	O(1)	O(1)	O(1)	Only touch top pointer
peek()	O(1)	O(1)	O(1)	Direct index/pointer read
isEmpty	O(1)	O(1)	O(1)	Single comparison
isFull	O(1)	N/A	N/A	Single comparison
Space Complexity				
Total	O(MAX)	O(n)	O(n)	n = current number of elements
Per element	data only	data + 1 pointer	data only	

* Vector `push_back` is *amortised* O(1); occasional O(n) when the internal buffer doubles.

A stack operates exclusively on its top element. No searching, no shifting, no traversal is ever required. Whether the stack contains ten elements or ten million, every push, pop, and peek performs a fixed number of instructions.

8. Quick Reference

8.1 Unified Method Names Across All Implementations

Operation	Array / LL	Vector	STL std::stack
Insert at top	push(val)	push(val)	push(val)
Remove top	pop()	pop()	pop()
Read top	peek()	peek()	top()
Check empty	isEmpty()	isEmpty()	empty()
Element count	size()	size()	size()

STL std::stack Caution

std::stack::pop() returns void — it does *not* return the removed element. Always call top() before pop():

```
#include <stack>
stack<int> s;
s.push(42);
int val = s.top();    // read first
s.pop();              // then remove
```

8.2 Implementation Comparison

```
//          1. Vector (no size limit, simplest)


class StackVec {
    vector<int> v;
public:
    void push(int val) { v.push_back(val); }
```

```
    int  pop()  { int t=v.back(); v.pop_back(); return t; }
    int  peek() const { return v.back(); }
    bool isEmpty() const { return v.empty(); }
};

//        2. Array (fixed capacity, cache-friendly)


class StackArr {
    int* arr; int top; int cap;
public:
    explicit StackArr(int c): cap(c), top(-1) { arr=new int[cap]; }
    ~StackArr() { delete[] arr; }
    void push(int v) { if(!isFull())  arr[++top]=v; }
    int  pop()       { if(!isEmpty()) return arr[top--]; return -1; }
    int  peek() const{ if(!isEmpty()) return arr[top]; return -1; }
    bool isEmpty() const { return top==-1; }
    bool isFull()  const { return top==cap-1; }
};

//        3. Linked List (unbounded, dynamic)


class StackLL {
    Node* top; int sz;
public:
    StackLL(): top(nullptr), sz(0) {}
    ~StackLL() { while(!isEmpty()) pop(); }
    void push(int v){ Node* n=new Node(v); n->next=top; top=n; ++sz; }
    int  pop()  { if(!top) return -1;
                  Node* t=top; int v=t->data;
                  top = top->next;          // assignment: advance top pointer
                  delete t; --sz; return v; }
    int  peek() const { if(!top) return -1; return top->data; }
    bool isEmpty() const { return top==nullptr; }
};
```

Listing 8.1: All three implementations side by side

8.3 Interview Pattern Reference

Problem Type	Technique	Examples
Matching / pairing	Single stack	Balanced brackets, HTML tags
Min/Max tracking	Auxiliary stack	Min-stack, max-stack
Next greater element	Monotonic stack	Stock span, daily temperatures
Reversal	Single stack	Reverse string, reverse linked list
Two-stack simulation	Two stacks	Queue using stacks, sort a stack
Expression evaluation	Single stack	Postfix, prefix evaluation
Histogram problems	Monotonic stack	Largest rectangle, rain water

8.4 Key Formulae

Array Stack Invariants

$$\text{isEmpty} \iff \texttt{top} = -1$$
$$\text{isFull} \iff \texttt{top} = \texttt{cap} - 1$$
$$\text{size} = \texttt{top} + 1$$
$$\texttt{push}(x) : \texttt{arr[++top] = x}$$
$$\texttt{pop}() : \texttt{return arr[top--]}$$

Linked-List Stack Invariants

isEmpty $\iff$ `top` $=$ `nullptr`

push: `newNode->next = top;  top = newNode;`
pop: `tmp = top;  top = top->next;  delete tmp;`
peek: guard `top != nullptr` first, then return `top->data`.

Two-Stack Invariants

Stack 1 empty $\iff$ `top1` $= -1$ Stack 2 empty $\iff$ `top2` $=$ `MAX`

Both full $\iff$ `top2` $-$ `top1` $= 1$

9. Summary of All Implementations and Problems

Topic	Key Concepts Covered
Stack using vector	Dynamic sizing; `push_back`/`pop_back`
Stack using array	Fixed capacity; `top` index; overflow/underflow guards
Stack using linked list	Dynamic allocation; node linking; `delete`; null-pointer guard
Two stacks in one array	Bidirectional growth; `top2-top1==1` full condition; `pop1()` fix
Reverse a stack	Three-pour method; odd reversal count
Adjacent duplicates	Top-comparison cancel technique
Balanced brackets	Push openers; match and pop closers; corrected C++ syntax
Decimal to binary	LIFO provides free reversal of remainder sequence
Postfix evaluation	`val2 OP val1` operand order; digit-to-int conversion

Stack Data Structure — Reference Complete

C++ · Array · Linked List · Vector · Two-Stack · Five Classic Problems